Amazoff

A Full-Stack Java Shopping App

Mahe Chen, Catherine Guo, Jacky Shen, Michael Gao

Specification

What Amazoff can do ...

Signup/Login Portal	Signup as a Buyer/Seller → Login as a Guest/Buyer/Seller → Deactivate an Account (Database API)
Personal Dashboard	View Profile Information → Perform Actions Based on Each User Class
Search Functionalities	Guest: Search Product → Buyer: Search: Purchase/add to cart→ Seller: Search/Sell Products (Walmart API)
Store Page (Seller)	An administrative panel that allows retailers to manage their product listings
Shopping Cart (Buyer)	Buyers can add products to their shopping carts, review items, and proceed to checkout
Order Tracker (Buyer)	Buyers can track orders

GITHUB:
https://github.com/lmposter/CSC207-Project

MORE INFORMATION:
https://github.com/lmposter/CSC207-Project/blob/main/README.md

Presenter: Mahe

Demo Login/Signup/Dashboard (All Users)

CSC207-Project1 main Main

Project

- CSC207-Project1 [CSC207-Project] C:\Users\pang
  - .idea
  - API
  - out
  - resources
  - src
    - app
    - data_access
    - entity
    - interface_adapter
    - use_case
      - allUser
      - create_product
      - login
      - orders
      - productDetails
      - search
      - signup
        - SignUpUserDataAccessInterface
        - SignUpUserInputBoundary
        - SignUpUserInputData
        - SignUpUserInteractor
        - SignUpUserOutputBoundary
        - SignUpUserOutputData
      - storePage
    - view
  - tests
  - .gitignore
  - API_TOKEN

Main.java

Run 'Main' Shift+F10

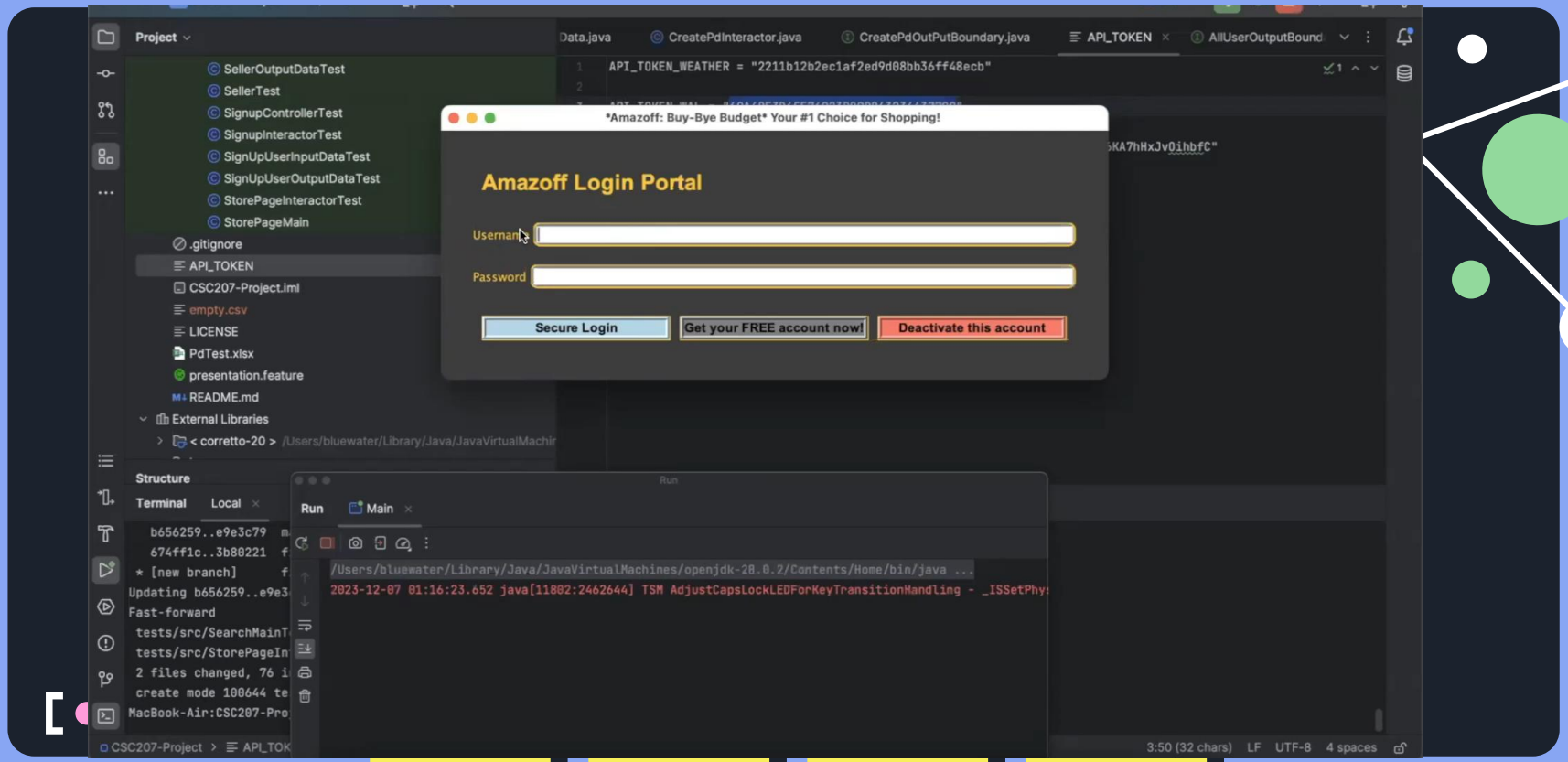
```java
package app;

import ...

Pumpkinmann +5
public class Main
{
    Pumpkinmann +5
    public static void main(String[] args) throws MalformedURLException
    {
        // Build the main program window, the main panel containing the
        // various cards, and the layout, and stitch them together.

        // The main application window.
        JFrame application = new JFrame( title: "*Amazoff: Buy-Bye Budget* Your #1 Choice for Shopping! ");
        application.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);

        CardLayout cardLayout = new CardLayout();

        // The various View objects. Only one view is visible at a time.
        JPanel views = new JPanel(cardLayout);
        application.add(views);

        // This keeps track of and manages which view is currently showing.
        ViewManagerModel viewManagerModel = new ViewManagerModel();
        new ViewManager(views, cardLayout, viewManagerModel);

        // The data for the views, such as username and password, are in the ViewModels.
        // This information will be changed by a presenter object that is reporting the
        // results from the use case. The ViewModels are observable, and will
        // be observed by the Views.
        LoginViewModel loginViewModel = new LoginViewModel();
```

CSC207-Project1 > src > app > Main > main 30:6 CRLF UTF-8 4 spaces

Presenter: Mahe

Demo

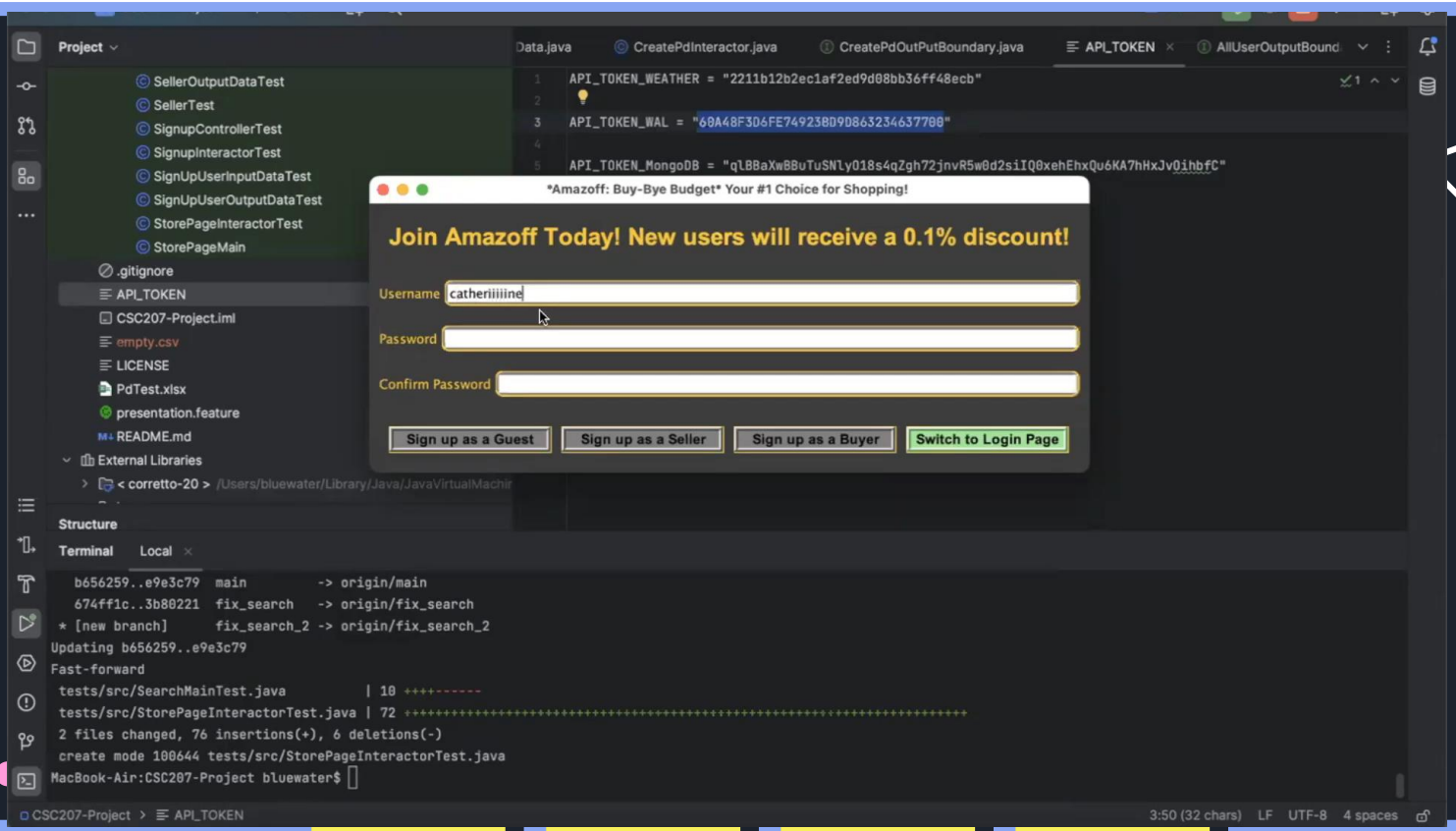
Buyer Use Cases



Jacky

Demo

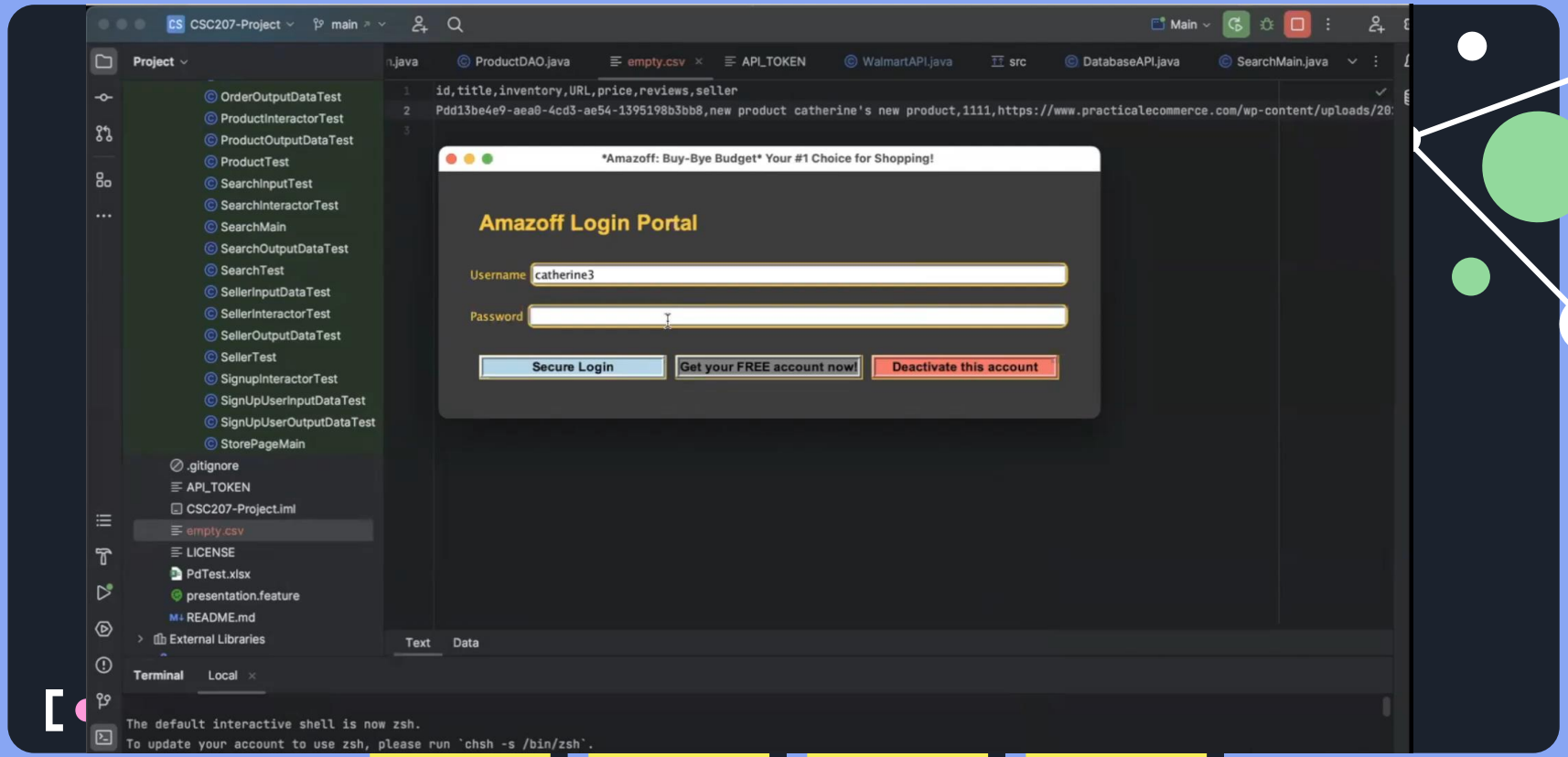
Guest Use Cases



Michael

Demo

Seller use cases



Catherine

API Usage

01

MongoDB API

Database

02

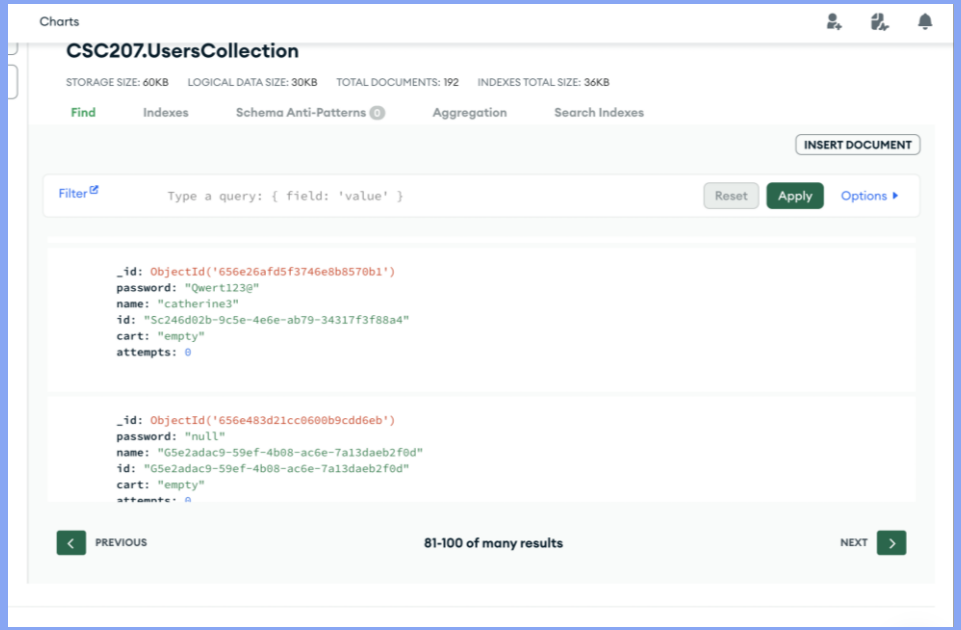
Walmart API

External products

For instance, we used the MongoDB API to store registered users and save products created by sellers, and the Walmart API to search for desired product listings.

Presenter: Catherine

Database API



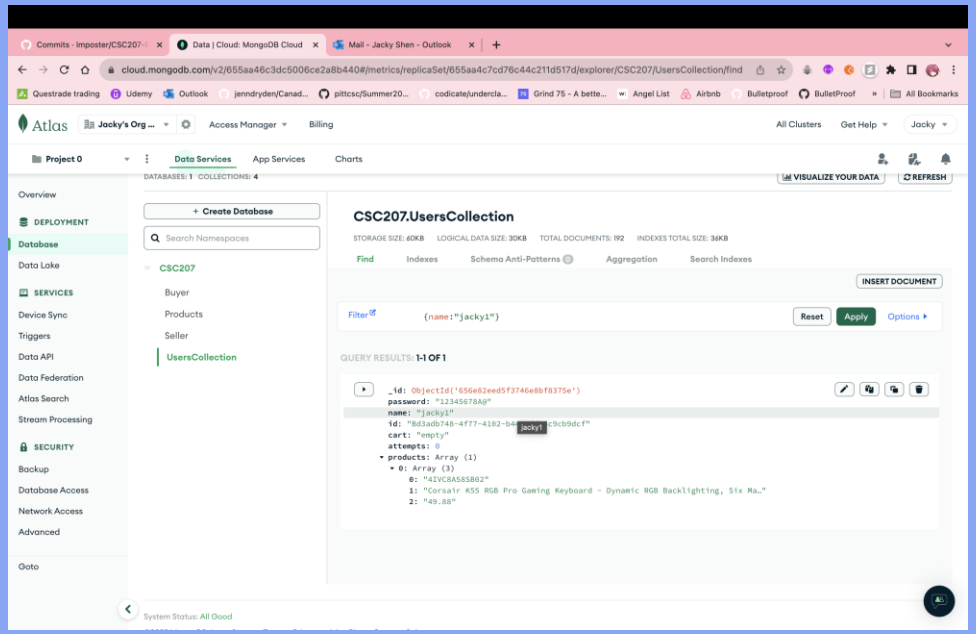
Link:
https://us-east-2.aws.data.mongodb-api.com/app/{username}/endpoint/data/v1/action/

Used for:

- Create & Get User
- Delete user
- Insert purchased products
- Get purchased product
- Increment attempt value
- Lock account

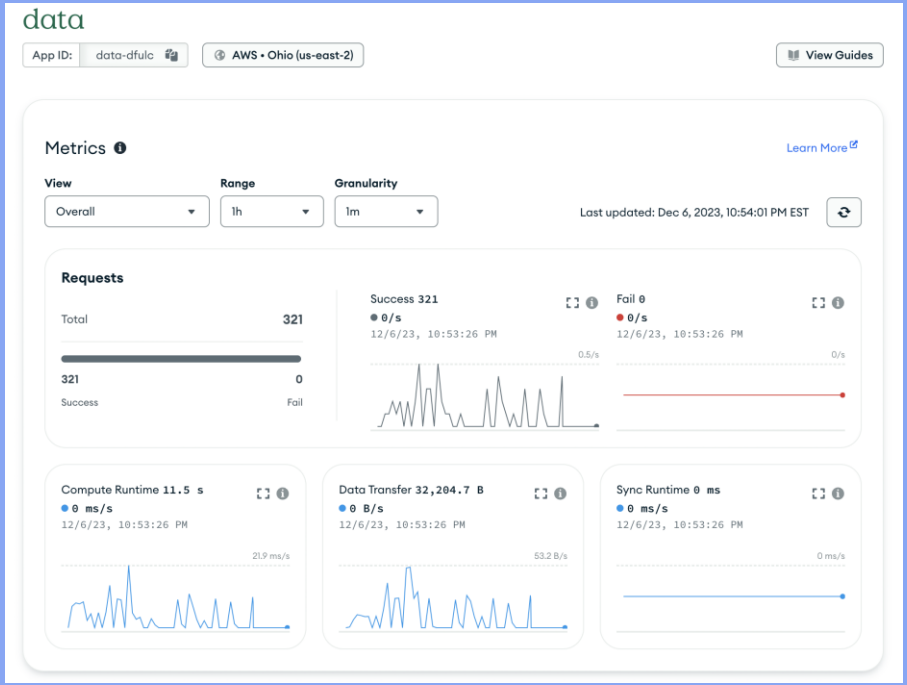
Jacky

Database API



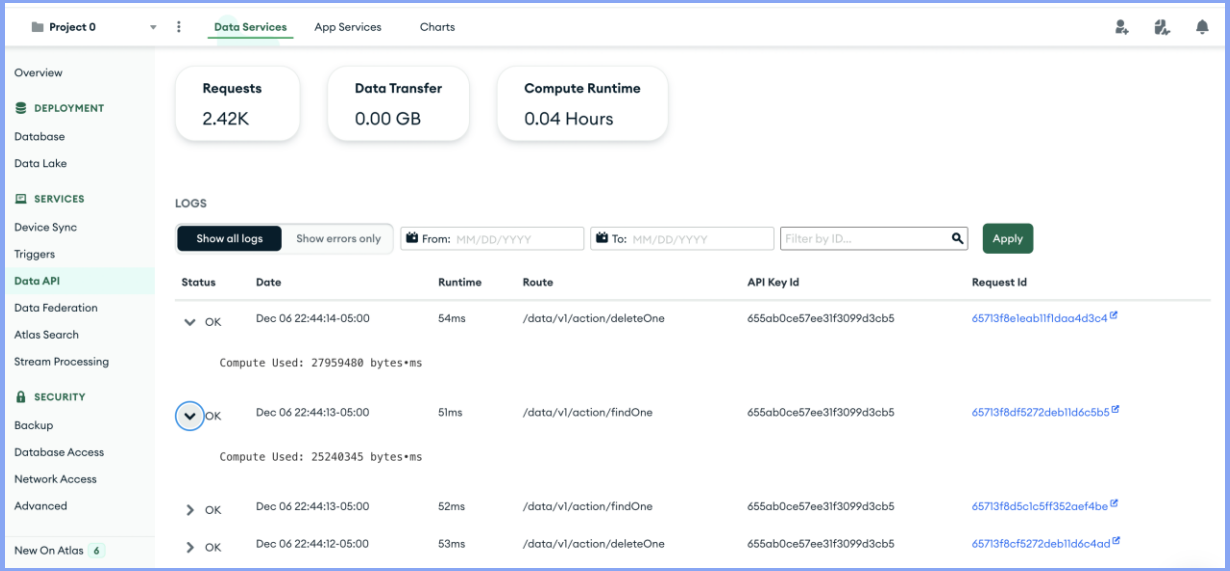
Jacky

Database API



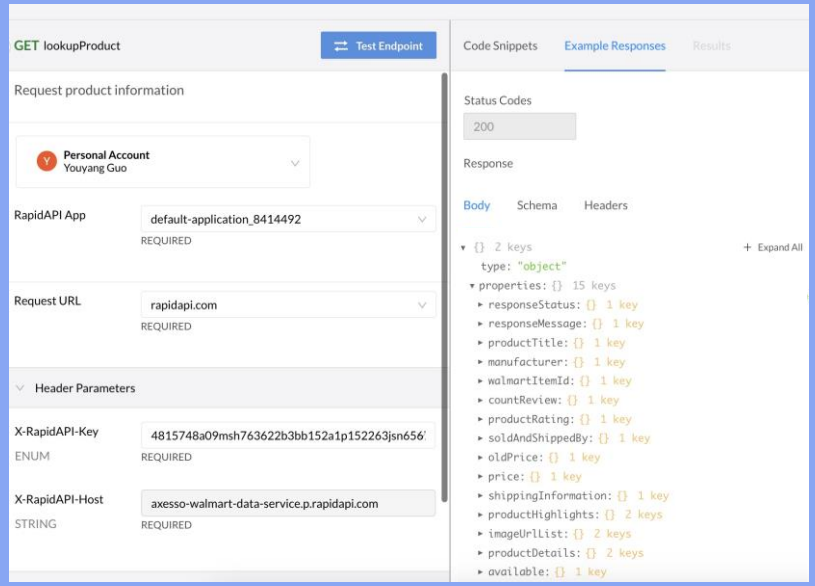
Jacky

Database API



Jacky

Walmart API



Link:
https://api.bluecartapi.com/request?api_key=apiToken&search_term=xxx&type=search

Used for:
- Title
- Price
- Inventory
- Review(stars)
- Image url

The products from Walmart will not be saved to local databases.

Catherine

SOLID Principles

1

Open-closed Principle

2

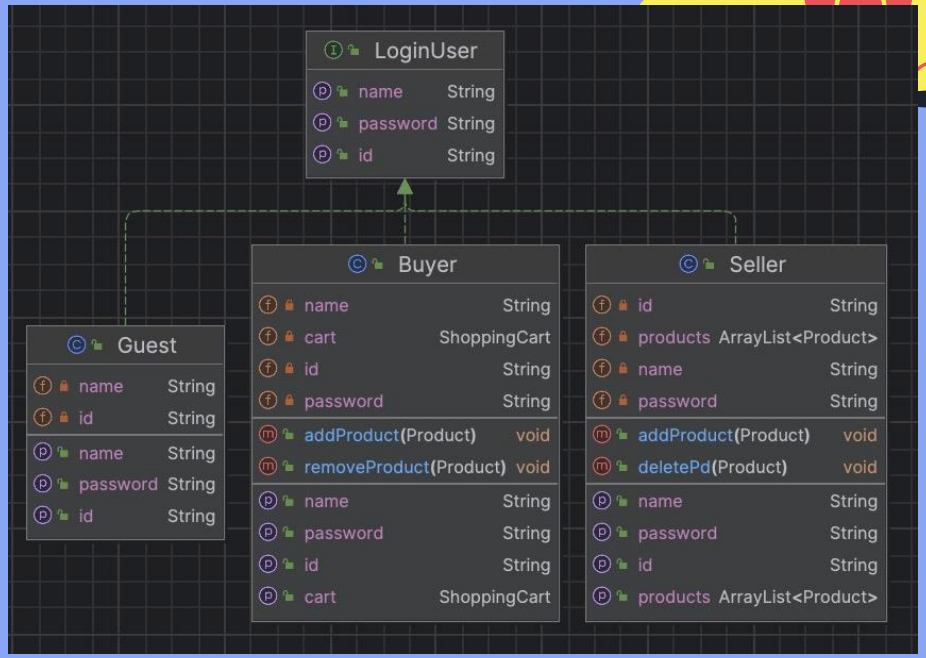
Single Responsibility Principle

3

Interface Segregation Principle

Open-Closed Principle

- Easy user role extension by inheriting **LoginUser** interface
- Basic requirements for a new user: name, password, ID
- Simple login capability for all valid users
- Dashboard access through inheriting **AllUser** interfaces

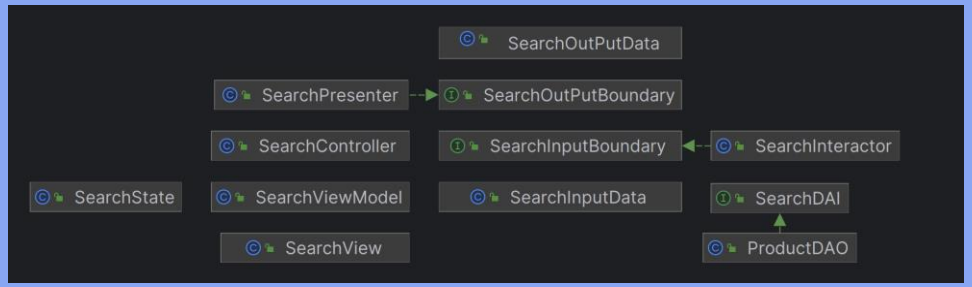


Jacky

Single Responsibility Principle

Search Use case: Each class has only one responsibility

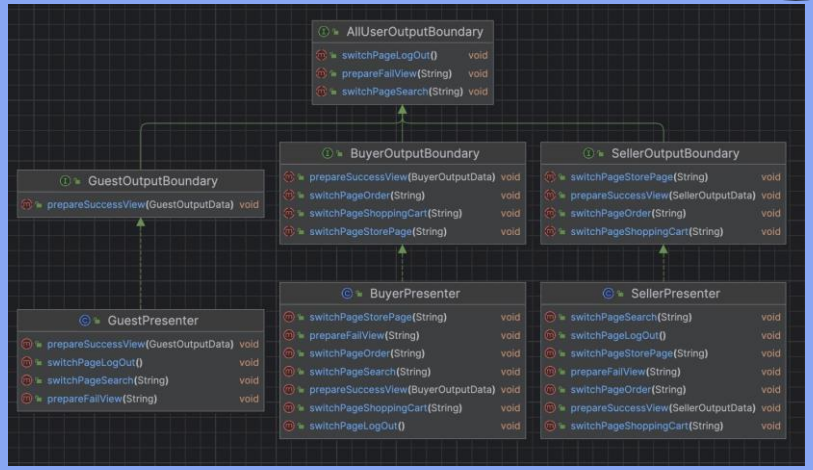
- SearchView: The searching bar for input
- SearchViewModel: Represent data in SearchView
- SearchState: Keep what the user types
- SearchInputData: What the user finally search
- SearchController: Pass InputData to SearchInteractor
- SearchInteractor: Get what the user is looking for
- ProductDAO: Access to available products
- SearchOutputData: What match the searching result
- SearchPresenter: Make it display the matching product



Michael

Interface Segregation Principle

- All user types (Guest, Buyer, Seller) extend **AllUser** interface
- Role-specific functionalities via additional interfaces
- Guests limited to logout and search functionality
- Buyers and Sellers granted extended options, including their own page controls



Catherine

Design Patterns

Factory Pattern

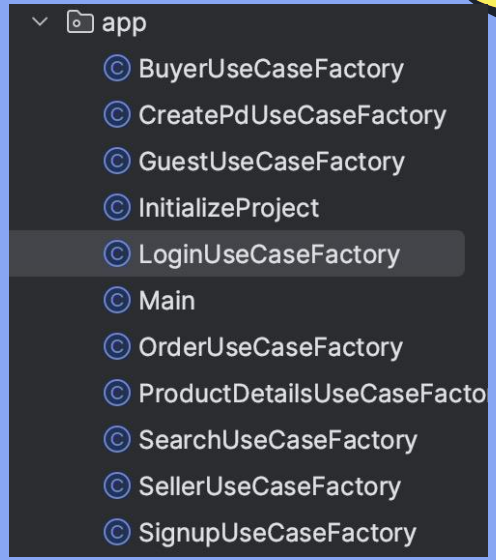
1

Strategy Design Pattern

2

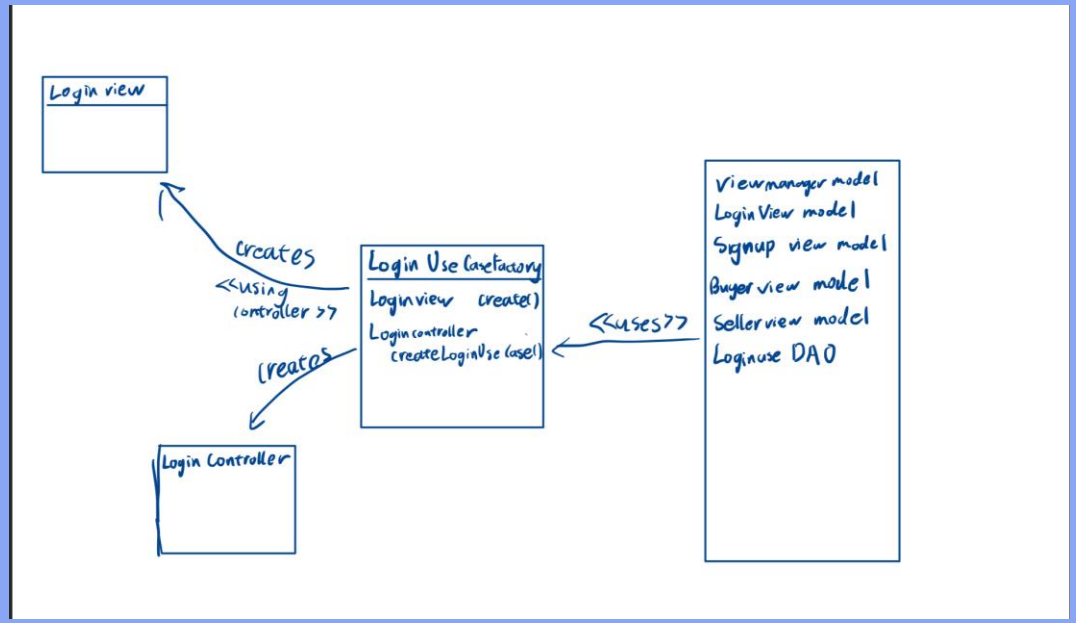
Factory Pattern

- Centralizes object creation logic for maintainability
- Encapsulates instantiation of user and order cases (e.g., buyer, seller, guest)
- Constructs LoginView with necessary components like LoginController
- Simplifies code adaptability and maintenance through centralized setup



Jacky

Factory Pattern



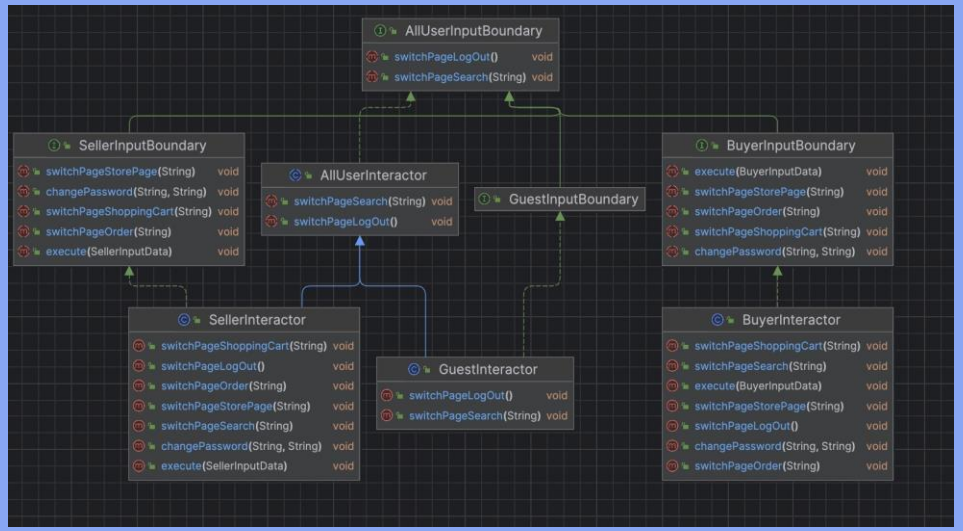
Jacky

Strategy design pattern

- (User logging in)
- Decides who is this
  (a seller? Or a customer?)
- Present related functionality

The login page doesn't know who is logging in. But when they click "log in", We know what kind of user this is and prepare different "strategy" for them.

This allows us to add more kind of users in future without making different log in page for each of them. We only make them a new dashboard.

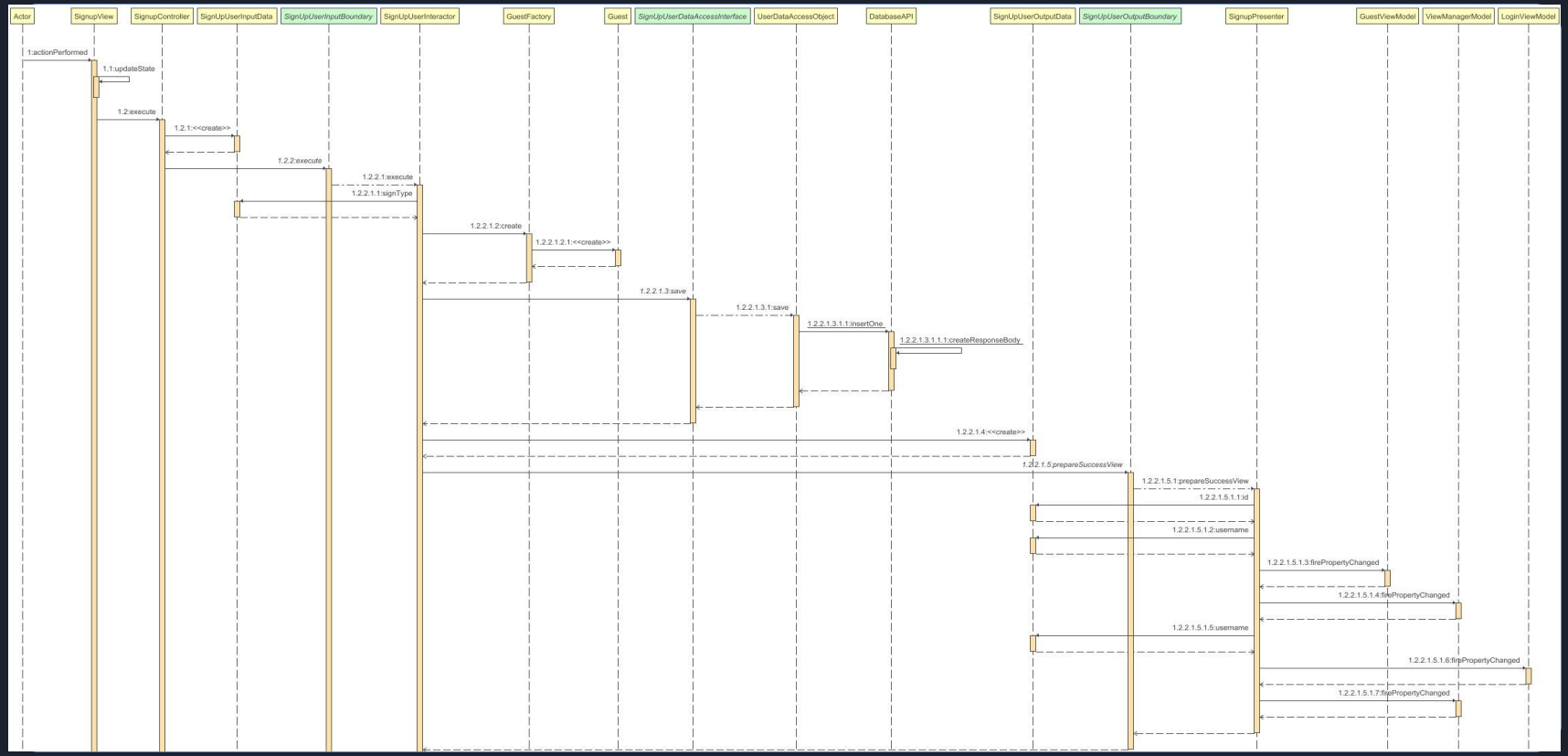


Michael

Clean Architecture

Use Case

Signup as a Guest



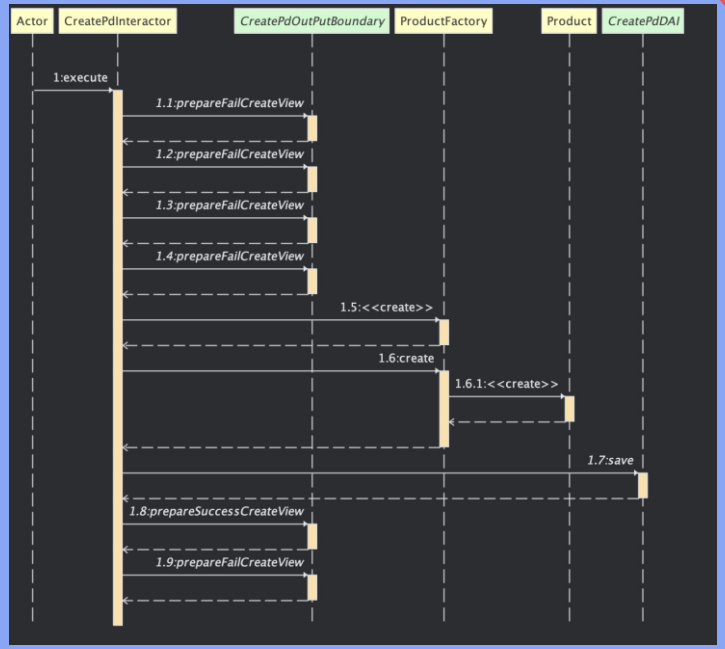
This use case creates a guest user based on user instruction, stores it in our database through the MongoDB API, notifies the LoginViewModel, passes the credentials to the GuestViewModel which is the dashboard for guests and finally change the active state to GuestViewModel via View Manager

Presenter: Mahe

Creating New Product (Seller)

Clean Architecture

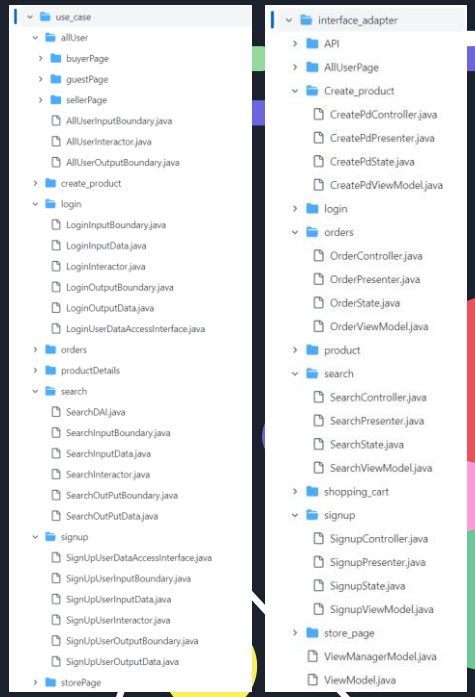
- Controller pass input to interactor
- Interactor whether input is valid
- Interactor create a product object
- Save product through interface
- Call presenter to notify seller



Michael

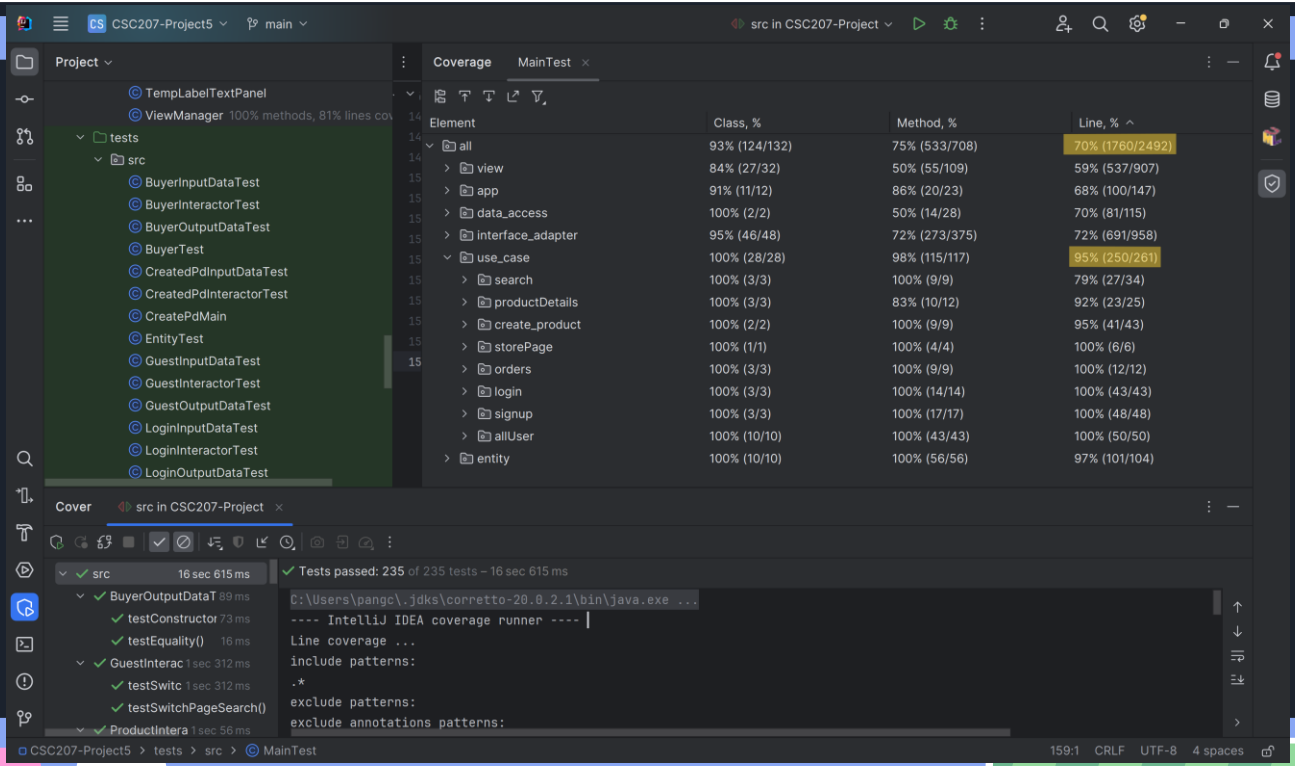
Code Organization

- The packages are organized based on Clean Architecture principles to enhance clarity and purpose.
- Within each package, further division by specific functionalities is achieved, as seen in the use_case and interface_adapter packages where subfolders are dedicated to individual use cases.
- The project was constructed using Maven, enabling us to incorporate additional dependencies, including org.json, org.mockito and com.squareup.okhttp.



Presenter: Mahe

Test Coverage



Untested: Some frontend code and style sheets, we can test them by simply visualizing them.
We approach the test with Mockito to mock users and Junit to test methods.

Presenter: Mahe

Thanks!

Does anyone have any questions?

GITHUB:
https://github.com/lmposter/CSC207-Project

CREDITS: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, and infographics & images by **Freepik**